

User Stories: Local Produce Exchange

1. Introduction

This document holds the user stories for the Local Produce Exchange. It is the companion to the team's use cases, the [Team Charter](#), and [project requirements](#). Each story names a user, the goal that user wants, and the benefit they get, followed by acceptance criteria.

Every user story has a "Source use case(s)" so each one is traceable back to the [use cases](#), which are in turn traceable back to the requirements and the charter. *Each story is one distinct functionality.*

Note: [Edge cases and permission rules are captured as scenarios inside the relevant story, not as separate stories, per the discussions in the \[#questions Discord channel\]\(#\).](#)

2. The user stories

Scenarios cover the normal path, and add edge or error, workflow, and permission scenarios only where the action needs them.

2.1 Authentication and account

US-01: Register with an invite token

- **Source use case:** UC-01 (Register with an invite token)
- **Priority:** high
- **Actor:** guest
- **Milestone:** Current scope (R1)

Story: As a guest, I want to register for an account using a valid invite token so that I can join the trusted community.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the guest holds a valid, unused invite token
- And the guest provides complete account details
- When the guest submits the registration form with the token
- Then a member account is created
- And the invite token is marked as used

Scenario 2 - edge / error path (bad token)

- Given the guest holds a token that is invalid, expired, or already used
- When the guest submits the registration form with that token
- Then the system rejects the registration
- And no account is created

Scenario 3 - edge / error path (bad details)

- Given the guest holds a valid, unused invite token
- And a required account detail is missing or malformed
- When the guest submits the registration form
- Then the system rejects the submission
- And shows a validation error

Notes / open questions / assumptions

- Token expiry rules are not yet defined. Confirm whether tokens expire and after how long.

US-02: Log in

- **Source use case:** UC-02 (Log in)
- **Priority:** high
- **Actor:** member
- **Milestone:** Current scope (R1)

Story: As a member, I want to log in to my account so that I can reach member features.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member has a registered account that is not suspended
- When the member submits correct credentials
- Then the system starts an authenticated session

Scenario 2 - edge / error path (wrong credentials)

- Given the member has a registered account
- When the member submits credentials that do not match
- Then the system denies login
- And no session is started

Scenario 3 - permission rule (suspended account)

- Given the member's account is suspended
- When the member submits correct credentials
- Then the system denies login
- And tells the member the account is suspended

US-03: Log out

- **Source use case:** UC-03 (Log out)
- **Priority:** medium
- **Actor:** member
- **Milestone:** Current scope (R1)

Story: As a member, I want to log out so that no one else can use my account on this device.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- When the member chooses to log out
- Then the system ends the session
- And returns the member to the public view

Scenario 2 - edge / error path (session already expired)

- Given the member's session has already expired
- When the member chooses to log out
- Then the system returns the member to the public view
- And no error is shown

Notes / open questions / assumptions

- None.

US-04: Invite a new member

- **Source use case:** UC-04 (Invite a new member)
- **Priority:** medium
- **Actor:** member
- **Milestone:** Current scope (R1)

Story: As a member, I want to create and share an invite token so that a new person can register and join the community.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in and active
- And the member is allowed to issue invites
- When the member chooses to create an invite
- Then the system generates a new, unused invite token
- And shows the token so the member can share it

Scenario 2 - permission rule (not allowed to invite)

- Given the member is not allowed to issue invites

- When the member attempts to create an invite
- Then the system denies the action
- And no token is created

Scenario 3 - permission rule (suspended member)

- Given the member is suspended
- When the member attempts to create an invite
- Then the system denies the action
- And no token is created

Notes / open questions / assumptions

- **ASSUMPTION:** the source documents require invite-only registration but do not say who issues invite tokens. Confirm whether any active member can invite, or only an admin. If the team does not decide the issuer, keep US-01 as core and treat US-04 as optional or pending.

US-05: View and update member profile

- **Source use case:** UC-05 (Manage member profile)
- **Priority:** medium
- **Actor:** member
- **Milestone:** Current scope (R1)

Story: As a member, I want to view and update my profile details so that my information stays current.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- When the member edits a profile field and saves
- Then the system validates the change
- And saves the updated profile

Scenario 2 - edge / error path (bad field)

- Given the member is logged in
- And a profile field is missing or malformed
- When the member saves the change
- Then the system rejects the change
- And shows a validation error

Scenario 3 - permission rule (editing another profile)

- Given the member is logged in
- When the member attempts to edit another member's profile
- Then the system denies the action
- And nothing changes

2.2 Discovery

US-06: Browse, search, and filter active listings

- **Source use case:** UC-06 (Browse, search, and filter active listings)
- **Priority:** high
- **Actor:** member
- **Milestone:** Current scope (R1)

Story: As a member, I want to browse, search, and filter active listings so that I can find items I want.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- When the member enters search terms or chooses filters such as category or dietary and allergen tags
- Then the system returns the active listings that match

Scenario 2 - edge / error path (no matches)

- Given the member is logged in
- When no active listings match the search or filters
- Then the system shows an empty result

US-07: View listing details

- **Source use case:** UC-07 (View listing details)
- **Priority:** high
- **Actor:** member
- **Milestone:** Current scope (R1)

Story: As a member, I want to see the full details of one listing so that I can decide whether to request it.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- And the listing is active
- When the member selects the listing
- Then the system shows its full details, including the quantity available, the tags, and the pickup window

Scenario 2 - edge / error path (no longer active)

- Given the member selects a listing that is no longer active
- When the system loads the listing
- Then the system tells the member the listing is unavailable

Notes / open questions / assumptions

- Photos are not part of the current listing detail view in this slice.

US-30: View another member's public profile

- **Source use case:** UC-26 (View another member's public profile)
- **Priority:** low
- **Actor:** member
- **Milestone:** Full Features (R2)

Story: As a member, I want to view another member's public profile and review history so that I can assess trustworthiness before requesting or approving an exchange

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- When the member opens another member's public profile
- Then the system shows that member's display name and their review history

Scenario 2 - edge / error path (own profile)

- Given the member opens their own profile through this view
- When the system loads the profile
- Then the system shows their own public profile the same way another member would see it.

Scenario 3 - permission rule (not logged in)

- Given a guest who is not logged in attempts to view a member profile
- When the system receives the request
- Then the system denies access

Notes / open questions / assumptions

- Private exchange details are not shown in this view.

2.3 Request queue

US-08: Submit a request for an item

- **Source use case:** UC-08 (Submit a request for an item)
- **Priority:** high
- **Actor:** recipient
- **Milestone:** Current scope (R1)

Story: As a recipient, I want to submit a request for a specified quantity from a listing so that my request joins the listing's queue and is handled in order.

Acceptance criteria

Scenario 1 - normal / valid path

- Given a listing is active
- And the listing has 10 units available
- When the recipient requests 3 units
- Then a request is created with status REQUESTED
- And the request is placed at the end of the listing's queue
- And the poster is notified of the new request

Scenario 2 - edge / error path (quantity exceeds available)

- Given a listing is active
- And the listing has 10 units available
- When the recipient requests 12 units
- Then the system rejects the request
- And no request is created

Scenario 3 - edge / error path (quantity must be positive)

- Given a listing is active
- When the recipient requests 0 units
- Then the system rejects the request
- And shows a validation error

Scenario 4 - edge / error path (duplicate open request)

- Given the recipient already has an open request on this listing
- When the recipient attempts to submit another request on the same listing
- Then the system prevents the duplicate request

Scenario 5 - permission rule (suspended member)

- Given the user submitting the request is suspended
- When the suspended user attempts to submit a request
- Then the system denies access
- And no request is created

Notes / open questions / assumptions

- This story models the requirements Sample User Story #1, adapted to the request-queue scope.

US-09: View the request queue for a listing

- **Source use case:** UC-09 (View the request queue for a listing)
- **Priority:** high
- **Actor:** poster
- **Milestone:** Current scope (R1)

Story: As a poster, I want to see the pending requests on my listing in the order they were received so that I can handle them fairly and in order.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the poster owns the listing
- When the poster opens the listing's request queue
- Then the system shows the pending requests oldest first, with the quantity requested and the quantity remaining

Scenario 2 - edge / error path (empty queue)

- Given the poster owns a listing with no pending requests
- When the poster opens the request queue
- Then the system shows an empty queue

Scenario 3 - permission rule (not the owner)

- Given a member who does not own the listing
- When that member attempts to view the listing's queue
- Then the system denies access

US-10: Approve or deny a pending request

- **Source use case:** UC-10 (Approve or deny the next request in the queue)
- **Priority:** high
- **Actor:** poster
- **Milestone:** Current scope (R1)

Story: As a poster, I want to approve or deny a pending request on my listing so that I control who receives my items, in order, without conflicts.

Acceptance criteria

Scenario 1 - normal / valid path (approve)

- Given the poster owns the listing
- And the request has status REQUESTED
- And the approved quantity will not exceed the remaining quantity
- When the poster approves the request
- Then the system sets the request status to APPROVED
- And reduces the remaining quantity
- And the recipient is notified

Scenario 2 - normal / valid path (deny)

- Given the poster owns the listing

- And the request has status REQUESTED
- When the poster denies the request
- Then the system sets the request status to DENIED
- And the recipient is notified

Scenario 3 - workflow rule (conflict prevention)

- Given the poster owns the listing
- And approving this request would push the approved quantity past the remaining quantity
- When the poster attempts to approve it
- Then the system rejects the approval
- And the status and remaining quantity do not change

Scenario 4 - workflow rule (wrong status)

- Given the request is not in REQUESTED status
- When the poster attempts to approve or deny it
- Then the system rejects the action
- And nothing changes

Scenario 5 - permission rule (not the listing owner)

- Given a member who does not own the listing
- When that member attempts to approve or deny the request
- Then the system denies access
- And nothing changes

Notes / open questions / assumptions

- A denial is final: the request cannot change status again.
- The poster handles requests in the order they are received.
- Approve and deny are two outcomes of one decision, so they are scenarios in this one story, not separate stories.

US-11: Withdraw a queued request

- **Source use case:** UC-11 (Withdraw a queued request)
- **Priority:** medium
- **Actor:** recipient
- **Milestone:** Current scope (R1)

Story: As a recipient, I want to withdraw a pending request I no longer need so that it leaves the queue and the poster knows.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the recipient owns a request with status REQUESTED

- When the recipient withdraws it
- Then the system sets the request status to CANCELLED
- And removes it from the queue
- And the poster is notified

Scenario 2 - workflow rule (not pending)

- Given the request is not in REQUESTED status
- When the recipient attempts to withdraw it
- Then the system rejects the withdrawal
- And nothing changes

Scenario 3 - permission rule (not the requester)

- Given a member who is not the requester for the request
- When that member attempts to withdraw it
- Then the system denies the action
- And nothing changes

Notes / open questions / assumptions

- Only a pending request can be withdrawn.

US-26: Cancel an approved request

- **Source use case:** UC-11 (Withdraw a queued request)
- **Priority:** medium
- **Actor:** recipient
- **Milestone:** Follow up scope (R2)

Story: As a recipient, I want to cancel a request that has been approved but not yet picked up, so that I can free up quantity for others if I no longer need the item

Acceptance criteria

Scenario 1 - normal / valid path

- Given the recipient owns a request with status APPROVED
- When the recipient cancels it
- The system sets the request to CANCELLED
- Restaurants the quantity back to the listing's quantity
- And the poster is notified

Scenario 2 - workflow rule (wrong status)

- Given the request is in terminal status PICKED_UP, DENIED, CANCELLED
- When the recipient attempts to cancel it
- The system rejects the action
- Nothing changes

Scenario 3 - permission rule (not the requester)

- Given a member who is not the requester for the request
- When that member attempts to cancel it
- The system denies the action
- Nothing changes

Notes / open questions / assumptions

- Restoring quantity on cancellation of an approved request should be confirmed with the team

2.4 Coordination

US-12: Send and read messages in the exchange thread

- **Source use case:** UC-12 (Send and read messages in the exchange thread)
- **Priority:** medium
- **Actor:** member
- **Milestone:** Current scope (R1)

Story: As a member who is the poster or the recipient for an exchange, I want to send and read messages in that exchange's private thread so that the two of us can coordinate the exchange.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is the poster or the recipient for the exchange
- When the member writes and sends a message
- Then the system saves the message to the thread
- And shows it to both parties

Scenario 2 - edge / error path (empty message)

- Given the member is a party to the exchange
- When the member sends an empty message
- Then the system rejects it

Scenario 3 - permission rule (not a party)

- Given a member who is not the poster or the recipient for the exchange
- When that member attempts to open the thread
- Then the system denies access

Notes / open questions / assumptions

- One thread links a poster and a recipient for a request.

2.5 Listings

US-13: Create a listing

- **Source use case:** UC-13 (Create a listing)
- **Priority:** high
- **Actor:** poster
- **Milestone:** Full features (R2)

Story: As a poster, I want to post a new listing with the required details so that other members can find and request the item.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the poster is a registered, active member
- When the poster enters a description, category, quantity available, dietary and allergen tags, and a pickup window, and saves
- Then the system validates the required details
- And saves the listing and makes it active

Scenario 2 - edge / error path (missing detail)

- Given the poster is creating a listing
- And a required detail is missing or invalid
- When the poster saves the listing
- Then the system rejects the listing
- And shows a validation error

Scenario 3 - permission rule (suspended member)

- Given the poster's account is suspended
- When the poster attempts to create a listing
- Then the system denies the action
- And no listing is created

Notes / open questions / assumptions

- Optional photo uploads during create are covered by US-25.
- The listing's remaining quantity starts equal to the quantity available.

US-14: Edit a listing

- **Source use case:** UC-14 (Edit a listing)
- **Priority:** medium
- **Actor:** poster
- **Milestone:** Full features (R2)

Story: As a poster, I want to update the details of one of my listings so that the information stays accurate.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the poster owns the listing
- When the poster changes the details and saves
- Then the system validates the changes
- And saves the updated listing

Scenario 2 - edge / error path (bad detail)

- Given the poster owns the listing
- And a changed detail is missing or invalid
- When the poster saves the change
- Then the system rejects the change
- And shows a validation error

Scenario 3 - permission rule (not the owner)

- Given the listing belongs to another member
- When the member attempts to edit it
- Then the system denies the action
- And nothing changes

Scenario 4 - permission rule (suspended member)

- Given the poster's account is suspended
- When the poster attempts to edit the listing
- Then the system denies the action
- And nothing changes

Notes / open questions / assumptions

- Adding, replacing, or removing photos during edit is covered by US-25.

US-15: Deactivate own listing

- **Source use case:** UC-15 (Deactivate own listing)
- **Priority:** medium
- **Actor:** poster
- **Milestone:** Full features (R2)

Story: As a poster, I want to deactivate one of my listings so that no new requests can be made on it.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the poster owns an active listing
- When the poster chooses to deactivate it

- Then the system marks the listing inactive
- And hides it from browsing

Scenario 2 - permission rule (not the owner)

- Given the listing belongs to another member
- When the member attempts to deactivate it
- Then the system denies the action
- And the listing stays active

Notes / open questions / assumptions

- Confirm what happens to requests already in progress on a listing that is deactivated. Assumption: existing requests continue as normal; only new requests are blocked.

2.6 Pickup and completion

US-16: Confirm pickup

- **Source use case:** UC-16 (Confirm pickup)
- **Priority:** high
- **Actor:** recipient
- **Milestone:** Full features (R2)

Story: As a recipient, I want to confirm that I picked up the item so that the exchange can be completed.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the recipient owns a request with status APPROVED
- When the recipient confirms pickup
- Then the system changes the request status from APPROVED to PICKED_UP
- And the poster is notified

Scenario 2 - workflow rule (wrong status)

- Given the request is not in APPROVED status
- When the recipient attempts to confirm pickup
- Then the system rejects the action
- And nothing changes

Scenario 3 - permission rule (not the requester)

- Given a member who is not the requester for the request
- When that member attempts to confirm pickup
- Then the system denies the action
- And nothing changes

US-17: Complete an exchange

- **Source use case:** UC-17 (Complete an exchange)
- **Priority:** high
- **Actor:** poster
- **Milestone:** Full features (R2)

Story: As a poster, I want to mark a picked-up exchange as completed so that both parties can leave a review.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the poster owns the listing
- And the request has status PICKED_UP
- When the poster marks the exchange complete
- Then the system changes the request status from PICKED_UP to COMPLETED
- And the recipient is notified

Scenario 2 - workflow rule (wrong status)

- Given the request is not in PICKED_UP status
- When the poster attempts to mark the exchange complete
- Then the system rejects the action
- And nothing changes

Scenario 3 - permission rule (not the listing owner)

- Given a member who does not own the listing
- When that member attempts to complete the exchange
- Then the system denies the action
- And nothing changes

2.7 Reviews

US-18: Leave a rating and review after completion

- **Source use case:** UC-18 (Leave a rating and review after completion)
- **Priority:** medium
- **Actor:** member
- **Milestone:** Full features (R2)

Story: As a member who took part in a completed exchange, I want to leave a short rating and review of the other party so that the two of us can build trust.

Acceptance criteria

Scenario 1 - normal / valid path

- Given an exchange has status COMPLETED
- And the member is the poster or the recipient for that exchange
- And the member has not already reviewed the other party
- When the member submits a rating and a short review
- Then the system saves the review
- And links it to the completed exchange
- And makes it visible to the reviewed party

Scenario 2 - workflow rule (not completed)

- Given an exchange has status REQUESTED, APPROVED, PICKED_UP, DENIED, or CANCELLED
- When the member attempts to submit a review
- Then the system rejects the review
- And no review is saved

Scenario 3 - edge / error path (duplicate review)

- Given the member has already reviewed the other party for this exchange
- When the member attempts to review again
- Then the system rejects the duplicate review

Scenario 4 - permission rule (non-participant)

- Given an exchange has status COMPLETED
- And the member is not the poster or the recipient for that exchange
- When the member attempts to submit a review
- Then the system denies access
- And no review is saved

US-19: View reviews for a completed exchange

- **Source use case:** UC-19 (View reviews for a completed exchange)
- **Priority:** low
- **Actor:** member
- **Milestone:** Full features (R2)

Story: As a member, I want to view the reviews left for a completed exchange so that I can see feedback, including any review left about me.

Acceptance criteria

Scenario 1 - normal / valid path

- Given an exchange has status COMPLETED
- And the member is the poster or the recipient for that exchange
- When the member opens the reviews for that exchange
- Then the system shows the reviews linked to it, including any review left about the viewing member

Scenario 2 - edge / error path (no reviews yet)

- Given a completed exchange has no reviews yet
- When a participant opens the reviews
- Then the system shows that there are no reviews yet

Scenario 3 - permission rule (non-participant)

- Given a member who is not a participant in the exchange
- When that member attempts to view the reviews
- Then the system denies access

Notes / open questions / assumptions

- None.

2.8 Notifications and dashboard

US-20: View status notifications

- **Source use case:** UC-20 (View status notifications)
- **Priority:** medium
- **Actor:** member
- **Milestone:** Full features (R2)

Story: As a member, I want to open and read in-app notifications about exchange status changes so that I know when I need to act.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- And the member has notifications
- When the member opens their notifications
- Then the system shows them newest first
- And the member can open the related exchange

Scenario 2 - edge / error path (none)

- Given the member has no notifications
- When the member opens their notifications
- Then the system shows an empty list

Notes / open questions / assumptions

- Email or SMS notifications are out of scope for this story set.

US-28: Mark a notification as read

- **Source use case:** UC-20
- **Priority:** low
- **Actor:** member
- **Milestone:** Full features (R2)

Story: As a member, I want to mark a notification as read so that I can track which status changes I have already seen.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- And the member has one or more unread notifications
- When the member opens an unread notification or marks it as read
- The system marks it as read
- And the system visually distinguishes between read/unread notifications

Scenario 2 - edge / error path (already read)

- Given the member is already marked read
- When the user attempts to mark as read again
- Then the system accepts silently
- And nothing changes

Notes / open questions / assumptions

- Confirm whether notifications should be deleted or marked as read for system.

US-21: View my dashboard and activity overview

- **Source use case:** UC-21 (View my dashboard and activity overview)
- **Priority:** high
- **Actor:** member
- **Milestone:** Full features (R2)

Story: As a member, I want a single dashboard of my activity so that I can see my active listings, incoming and outgoing requests, and exchange history in one place.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the member is logged in
- When the member opens their dashboard
- Then the system shows their active listings, incoming requests, outgoing requests, and exchange history, grouped by status
- And each item links to the action that fits its status (approve or deny a request, withdraw a request, deactivate a listing, confirm pickup, complete an exchange)

Scenario 2 - edge / error path (no activity)

- Given the member has no activity yet
- When the member opens their dashboard
- Then the system shows empty groups

2.9 Admin

US-22: Suspend a user

- **Source use case:** UC-22 (Suspend a user)
- **Priority:** medium
- **Actor:** admin
- **Milestone:** Full features (R2)

Story: As an admin, I want to suspend a member account so that a misbehaving user can no longer log in or take member actions.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the admin is logged in with admin rights
- And the target account exists
- When the admin suspends the account
- Then the system marks the account suspended
- And the account can no longer log in or take member actions

Scenario 2 - permission rule (non-admin)

- Given a user without admin rights
- When that user attempts to suspend an account
- Then the system denies the action
- And nothing changes

US-27: Unsuspend a user

- **Source use case:** UC-22 (Suspend a user)
- **Priority:** medium
- **Actor:** admin
- **Milestone:** Full features (R2)

Story: As an admin, I want to unsuspend a member account so that a user who was incorrectly flagged or has taken steps to remediate behavior can have their account reinstated

Acceptance criteria

Scenario 1 - normal / valid path

- Given the admin is logged in with admin rights
- And the target account exists and is currently suspended
- When the admin reinstates the account
- Then the system marks the account as active
- And the account can then longer log in or take member actions

Scenario 2 - permission rule (non-admin)

- Given a user without admin rights
- When that user attempts to unsuspend an account
- Then the system denies the action
- And nothing changes

US-23: Deactivate a listing as admin

- **Source use case:** UC-23 (Deactivate a listing as admin)
- **Priority:** medium
- **Actor:** admin
- **Milestone:** Full features (R2)

Story: As an admin, I want to deactivate any listing so that I can hide it from browsing without deleting its audit history.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the admin is logged in with admin rights
- And the listing exists
- When the admin deactivates the listing
- Then the system hides the listing from browsing
- And keeps its audit history

Scenario 2 - permission rule (non-admin)

- Given a user without admin rights
- When that user attempts to deactivate a listing as admin
- Then the system denies the action
- And nothing changes

US-24: Generate basic reports

- **Source use case:** UC-24 (Generate basic reports)
- **Priority:** low
- **Actor:** admin
- **Milestone:** Full features (R2)

Story: As an admin, I want to generate a basic report about system activity so that I can see how the exchange is being used.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the admin is logged in with admin rights
- When the admin generates a report
- Then the system computes the report
- And shows it to the admin

Scenario 2 - permission rule (non-admin)

- Given a user without admin rights
- When that user attempts to generate a report
- Then the system denies the action

US-29: View member profile as admin

- **Source use case:** UC-25 (View member profile as admin)
- **Priority:** medium
- **Actor:** admin
- **Milestone:** Full features (R2)

Story: As an admin, I want to search and view a member's full account details so that I can find the right account before taking administrative action.

Acceptance criteria

Scenario 1 - normal / valid path

- Given the admin is logged in with admin rights
- When the admin searches for a member by name or email
- Then the system returns matching members with their account status
- And the admin can select a member to view their full account details and active/suspension status

Scenario 2 - edge / error path (no matches)

- Given the admin searches for a member by name or email
- And no members match
- When the system processes the search
- The system shows an empty result

Scenario 3 - edge / error path (suspended account)

- Given the admin selects a suspended account
- When the system loads the account details
- The system shows account details with suspended status

- And makes the reinstate action available

Scenario 4 - permission rule (non-admin)

- Given a user without admin rights
- When that user attempts to access the member profile view
- The system denies access

2.10 Listing photos

US-25: Add and manage listing photos

- **Source use case:** UC-13 (optional photos during create), UC-14 (add, replace, or remove photos during edit)
- **Priority:** low
- **Actor:** poster
- **Milestone:** Full features (R2)

Story: As a poster, I want to add, replace, and remove photos on my listing so that members can see what the item looks like.

Acceptance criteria

Scenario 1 - normal / valid path (add during create or edit)

- Given the poster owns the listing
- When the poster adds one or more photos and saves
- Then the system stores the photos with the listing

Scenario 2 - normal / valid path (replace or remove)

- Given the poster owns a listing that already has photos
- When the poster replaces or removes a photo and saves
- Then the system updates the listing's photos

Scenario 3 - permission rule (not the owner)

- Given the listing belongs to another member
- When the member attempts to change its photos
- Then the system denies the action
- And the photos do not change

Notes / open questions / assumptions

- Photos are optional on a listing. Confirm allowed file types and a maximum size or count before building.

3. Assumptions and open questions

These items are not yet approved by the team. Each one should be confirmed or changed before the requirements are locked. They are also noted inline in the affected stories.

ID	Item	Affected stories	Status
-----------	-------------	-------------------------	---------------
